Домашнее задание

Содержание

- Задача 1. Реализация методов СЛАУ для уравнения Пуассона.
- Задача 2. Метод Шульца (минимальных невязок) для вычисления обратной матрицы.
- Задача 3. Метод вращений Гивенса и QR-разложение.
- Задача 4. Асимпотика метода Гивенса.
- Задача 5. SVD для разреженных матриц и МНК.
- Задача 6*. Итерационное SVD.

Задача 1. Реализация методов СЛАУ для уравнения Пуассона.

```
import numpy as np
from scipy.linalg import block_diag
```

Рассмотрим систему линейных уравнений, матрица правой части которой является «ленточной» и имеет следующую структуру: ненулевые элементы расположены на трех центральных диагонялях и на двух «крыльях». Матрицы такой структуры возникают, например, при решении задачи на нахождение электростатического потенциала $\phi(x, y)$, создаваемого двумерным распределением заряда $\rho(x, y)$ при дискретизации на сетке уравнения Пуассона

$$\Delta\phi = -4\pi\rho.$$

(детали см. напр. А.А. Самарский, А.В. Гулин, Численные методы, Ч. 3 гл. 1, параграф 1). Размер матрицы растет с уменьшением шага сетки h как $O\left(1/h^2\right)$. Таким образом, приходится иметь дело с разреженными матрицами огромного размера. Матрицу m создадим следующим образом:

Back to top

```
n = 5

# Диагональный блок
a = np.zeros((n-1, n-1))

idx = np.arange(n-1)

a[idx, idx] = -4
a[idx[:-1], idx[:-1]+1] = 1
a[idx[1:], idx[1:]-1] = 1

# собираем блочно-диагональную `m`
m = block_diag(a, a, a, a, a)

# заполняем "крылья"
idx = np.arange(m.shape[0])

m[idx[:-n+1], idx[:-n+1] + n-1] = 1
m[idx[n-1:], idx[n-1:] - n+1] = 1
```

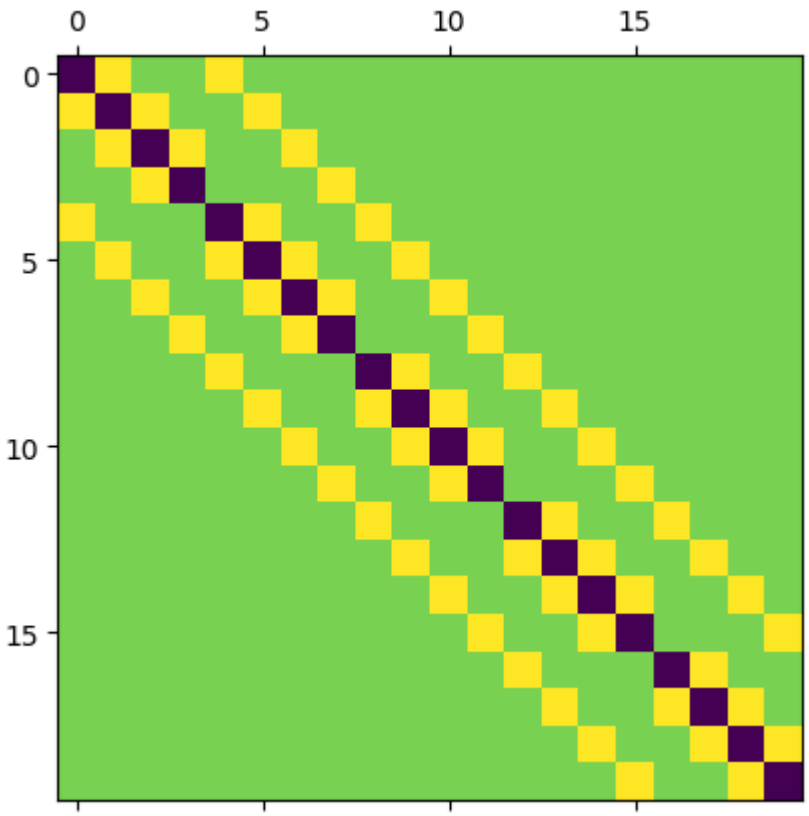
```
with np.printoptions(linewidth=99):
    print(m)
```

```
[[-4.  1.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 [ 1. -4.  1.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 [ 0.  1. -4.  1.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 [ 0.  0.  1. -4.  0.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 [ 1.  0.  0.  0. -4.  1.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 [ 0.  1.  0.  0.  1. -4.  1.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.
 [ 0.  0.  1.  0.  0.  1. -4.  1.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.
 [ 0.  0.  0.  1.  0.  0.  1. -4.  0.  0.  0.  1.  0.  0.  0.  0.  0.  0.
 [ 0.  0.  0.  0.  1.  0.  0.  0. -4.  1.  0.  0.  1.  0.  0.  0.  0.  0.
 [ 0.  0.  0.  0.  0.  1.  0.  0.  1. -4.  1.  0.  0.  1.  0.  0.  0.  0.
 [ 0.  0.  0.  0.  0.  0.  1.  0.  0.  1. -4.  1.  0.  0.  1.  0.  0.  0.
 [ 0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  1. -4.  0.  0.  0.  1.  0.  0.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  0. -4.  1.  0.  0.  1.  0.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  1. -4.  1.  0.  0.  1.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  1. -4.  1.  0.  0.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  1. -4.  0.  0.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  0. -4.  1.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  1. -4.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  1.
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.
```

```
import matplotlib.pyplot as plt
plt.matshow(m)
```

Back to top

```
<matplotlib.image.AxesImage at 0x7e48f9085480>
```



Зададим вектор правой части системы

```
b = np.zeros(m.shape[0])
b[m.shape[0]//2] = -1
```

Составить функцию, вычисляющую решение системы уравнений $mx = b$ описанными ниже методами с заданной точностью ϵ. Построить в каждом случае зависимость числа итераций, требуемых для достижения заданной точности, от ϵ.

Первый метод.

- Создать эту же матрицу с помощью scipy.sparse.csc_matrix.
- Создать эту же матрицу с помощью scipy.sparse.diags.
- Создать эту же матрицу с помощью scipy.sparse.spdiags.
- Решите полученную систему с помощью встроенного метода прогонки для полосатых матриц (составленных из диагоналей) scipy.linalg.solve_banded.

Второй метод.

- Решите полученную систему с помощью встроенного прямого метода для полосатых матриц (составленных из диагоналей) scipy.sparse.linalg.spsolve.

Back to top

Третий метод.

- Напишите распараллеленый метод Зейделя, ускорив его с помощью Numba. Указание: достаточно взять из файла реализованный метод jacobi_solve_numb и слегка его изменить. Распараллеливание было там же, с помощью series_timed_numba.

Задача 2. Метод Шульца (минимальных невязок) для вычисления обратной матрицы.

Реализуйте итерационный метод Шульца для вычисления обратной матрицы.

Описание метода.

Введем невязку $\Psi^{(k)} = E - AX^{(k)}$, где $X^{(k)}$ - приближение с номером k.

Рассмотрим
$(E - \Psi^{(k)})^{-1} = E + (\Psi^{(k)}) + (\Psi^{(k)})^2 + (\Psi^{(k)})^3 + \ldots = (AX^{(k)})^{-1} = (X^{(k)})^{-1}A$

Умножим обе части на $X^{(k)}$:

$$X^{(k)}(\sum_{i=0}^{\infty}(\Psi^{(k)})^i) = A^{-1}$$

К сожалению, считать бесконечный ряд мы не можем. Таким образом мы можем ввести итерацию :

$$X^{(k+1)} = X^{(k)}(\sum_{i=0}^{m}(\Psi^{(k)})^i)$$

Где m - порядок метода.

Получается порядок действий:

- Задать начальное приближение, порядок метода и необходимую точность.
- Вычислить невязку : $\Psi^{(k)} = E - AX^{(k)}$
- Проверить норму невязки на точность
- Найти следующее приближен Back to top ие: $X^{(k+1)} = X^{(k)}(\sum_{i=0}^{m}(\Psi^{(k)})^i)$

Замечание:

Данный метод сходится при норме первой невязки меньше единицы.

Задача 3. Метод вращений Гивенса и QR-разложение.

Введём матрицу $n \times n$ поворота Гивенса (она единичная, за исключением строк i, k , в которых только два ненулевых элемента):

$$G(k,l,\theta) = \begin{array}{cc} \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & c & \cdots & s & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & -s & \cdots & c & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} & \begin{array}{c} \\ \\ k \\ \\ l \\ \\ \\ \end{array} \end{array}$$

где $c = \cos\theta$ и $s = \sin\theta$ для некоторого θ. Таким образом, матрица поворота Гивенса $G(k,l,\theta)$ - ортогональна.

Пусть есть произвольная $A \in \mathbb{R}^{n\times n}$.

1. Какие элементы матрицы A поменяются при домножении $G(k,l,\theta)^T$ слева на A?
2. Какие k, l, s, c надо выбрать, чтобы по гарантировать зануление элемента $i, j\,(i > j)$ после такого перемножения? Не забывайте, что G должна быть ортогональной.
3. Учитывая ортогональность всех матриц Гивенса (а домножение ортогональной на ортогональную есть ортогональная), **постройте алгоритм построения QR-разложения матрицы** A.

Подсказка 📺

Даже не для квадратной $A \in \mathbb{R}^{4\times 3}$ рассматривая $G \in \mathbb{R}^{4\times 4}$ хотим нечто такое:

Back to top

$$
\begin{aligned}
A = & \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G_1^T(3,4)} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_2^T(2,3)} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_3^T(1,2)} \\
\longrightarrow & \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_4^T(3,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{G_5^T(2,3)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{G_6^T(3,4)} \\
\longrightarrow & \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R
\end{aligned}
$$

При этом,

$$Q = G_1(3,4)G_2(2,3)G_3(1,2)G_4(3,4)G_5(2,3)G_6(3,4)$$

$$A = QR$$

Задача 4. Асимпотика метода Гивенса.

Покажите, что при больших n метод Гивенса требует $\approx 4n^3/3$ действий умножения. Докажите, что метод Гаусса требует примерно в 4 раза меньше.

.

Задача 5. SVD для разреженных матриц и МНК.

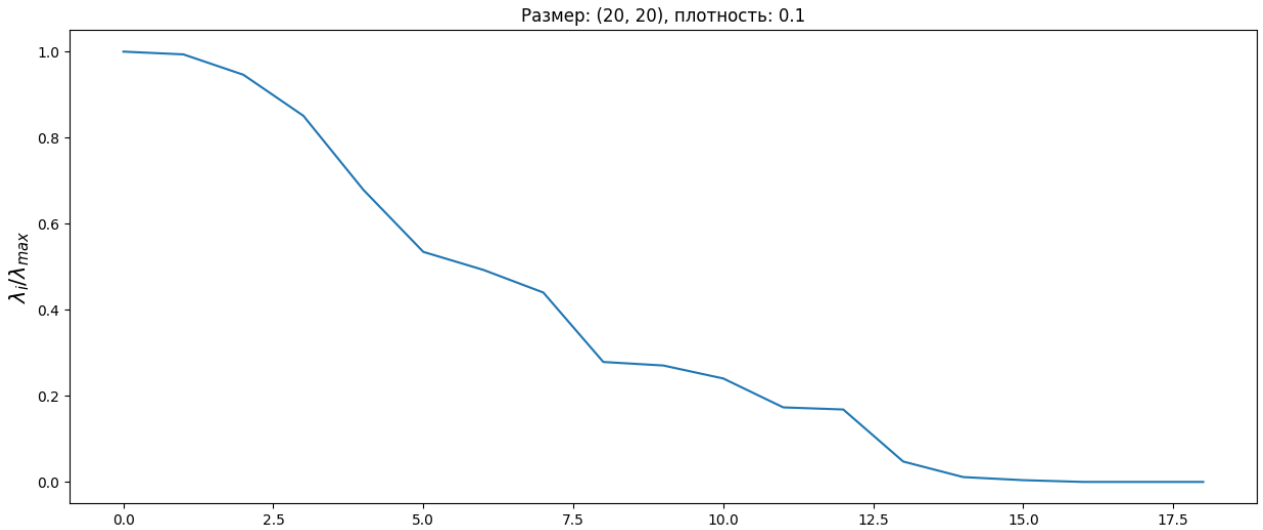
Для разреженных матриц SVD считают упрощённым методом. Для таких матриц обычно нужны не все сингулярные числа и вектора, а только первые k. Для этого используются специальные алг[illegible]осто вызовём функцию из модуля scipy.sparse.linalg

Back to top

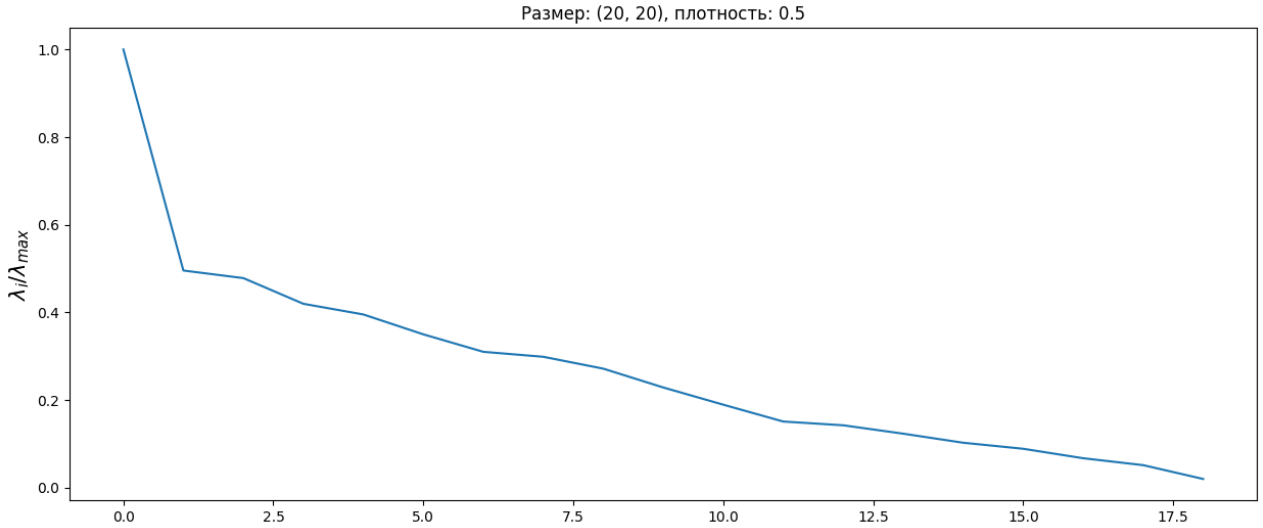
Чтобы проиллюстрировать это, напишем программу, которая генерирует случайные

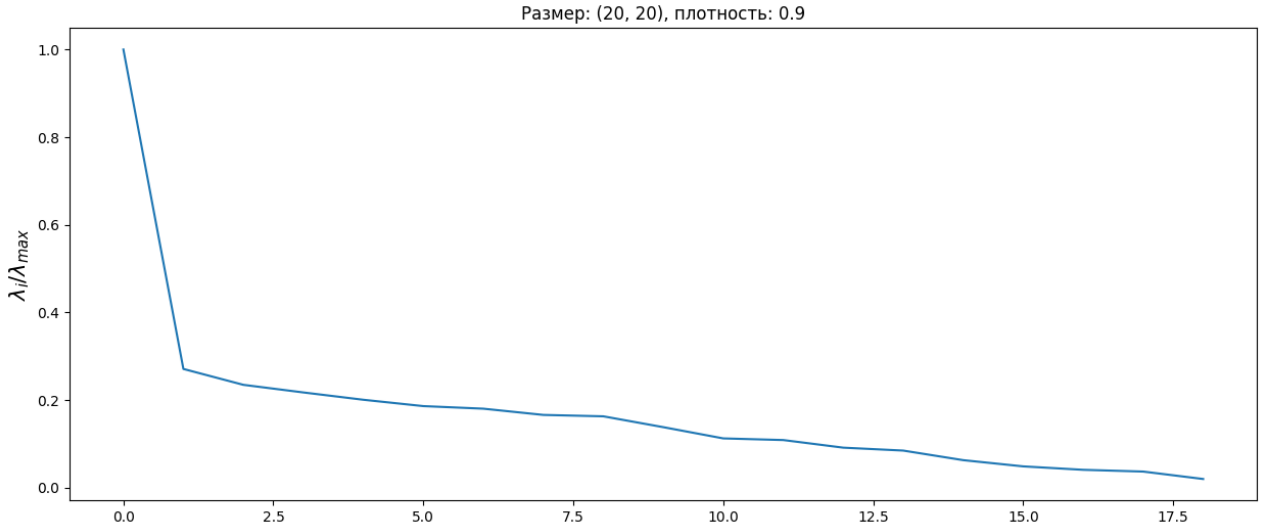
разреженные матрицы и строит график убывания сингулярных чисел.

```
import scipy as sp
import matplotlib.pyplot as plt
import numpy as np

def singular_values_attenuation_in_sparse(shape:tuple, density:float):
    '''
    Исследует убывание сингулярных чисел в разреженных матрицах
    размера shape и с плотностью density (от 0 до 1)
    '''
    sing_values_sum = np.zeros(min(shape)- 1 )

    sample_size = 100

    for _ in range(sample_size):
        sparse = sp.sparse.rand(shape[0], shape[1],
                                density=density,
                                random_state=42)
        sing_values_sum += sp.sparse.linalg.svds(sparse, k=min(shape)-1,
                                                 return_singular_vectors=False)
    plt.figure(figsize=(15, 6))
    plt.title(f'Размер: {shape}, плотность: {density}')
    plt.ylabel(r'$\lambda_i / \lambda_{max}$', fontsize=15)

    mean_vals = sing_values_sum[::-1] / sample_size

    plt.plot(mean_vals / mean_vals[0])

size = 20
singular_values_attenuation_in_sparse((size, size), 0.1)
singular_values_attenuation_in_sparse((size, size), 0.5)
singular_values_attenuation_in_sparse((size, size), 0.9)
```



Back to top





Пусть $A = U\Sigma V^*$ SVD для матрицы A. Тогда псевдообратная матрица равна:

$$A^\dagger = V\Sigma^\dagger U^*$$

где $\Sigma^\dagger$ состоит из обращённых ненулевых сингулярных чисел матрицы A. Действительно,

$$A^\dagger = \lim_{\alpha\to 0}\left(\alpha I + A^*A\right)^{-1}A^* = \lim_{\alpha\to 0}\left(\alpha VV^* + V\Sigma^2V^*\right)^{-1}V\Sigma U^* = \lim_{\alpha\to 0}\left(V\left(\alpha I + \right.\right.$$
$$= V\lim_{\alpha\to 0}\left(\alpha I + \Sigma^2\right)^{-1}\Sigma U^* = V\Sigma^\dagger U^*$$

- Вы можете проверить, что $\Sigma^\dagger$ состоит из обращённых ненулевых сингулярных чисел
- Если сингулярные числа малы. их можно не обращать, а просто приравнять к нулю. Это даст решение, мен Back to top льное к шуму в правой части.

Псевдообратная матрица решает задачу минимизации методом наименьших квад-

ратов. Однако она не является самым оптимальным способом решить эту задачу. Более эффективно использовать QR-разложение. Если A имеет полный ранг, то

$$x = A^{\dagger}b = (A^*A)^{-1}A^*b = ((QR)^*(QR))^{-1}(QR)^*b = (R^*Q^*QR)^{-1}R^*Q^*b = R$$

Таким образом, необходимо решить следующую квадратную систему: $Rx = Q^*b$

- R верхнетреугольная
- Решение требует $\mathcal{O}\left(n^2\right)$ операций
- Более устойчивый способ, чем использование псевдообратной матрицы напрямую.

Выполните следующие задания:

- Напишите программу, которая вычисляет псевдообратную матрицу с помощью SVD. Исследуйте, что происходит с числом обусловленности.
- Напишите решение задачи МНК с помощью вычисления псевдобратной матрицы через SVD
- Реализуйте QR-разложением и сравните с работой алгоритма из предыдущего пункта на случайных матрицах. Отдельно проведите сравнение на разреженных матрицах и плотных, выбирая разное значение density.

Задача 6*. Итерационное SVD.

Напишите свои собственные функции вычисления SVD простым итерационным методом и с помощью метода вращений Якоби. Сравните их работу с работой библиотечной функции numpy.linalg.svd.

Описание методов.

1. Простой итерационный алгоритм

Основная процедура — поиск наилучшего приближения произвольной $m \times n$ матрицы $X = (x_{ij})$ матрицей вида $b \otimes a = (b_i a_j)$ (где b — m-мерный вектор, а a — n-мерный вектор) методом наименьших квадратов:

Back to top

$$F(b,a)=\frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}(x_{ij}-b_ia_j)^2\to\min$$

Решение этой задачи дается последовательными итерациями по явным формулам. При фиксированном векторе $a=(a_j)$ значения $b=(b_i)$, доставляющие минимум форме $F(b,a)$, однозначно и явно определяются из равенств $\partial F/\partial b_i=0$:

$$\frac{\partial F}{\partial b_i}=-\sum_{j=1}^{n}(x_{ij}-b_ia_j)a_j=0;b_i=\frac{\sum_{j=1}^{n}x_{ij}a_j}{\sum_{j=1}^{n}a_j^2}$$

Аналогично, при фиксированном векторе $b=(b_i)$ определяются значения $a=(a_j)$:

$$a_j=\frac{\sum_{i=1}^{m}b_ix_{ij}}{\sum_{i=1}^{m}b_i^2}$$

В качестве начального приближения вектора a возьмем случайный вектор единичной длины, вычисляем вектор b, далее для этого вектора b вычисляем вектор a и т. д. Каждый шаг уменьшает значение $F(b,a)$. В качестве критерия остановки используется малость относительного уменьшения значения минимизируемого функционала $F(b,a)$ за шаг итерации $(\Delta F/F)$ или малость самого значения F.

В результате для матрицы $X=(x_{ij})$ получили наилучшее приближение матрицей P_1 вида $b^1\otimes a^1=(b_i^1a_j^1)$ (здесь верхним индексом обозначен номер итерации). Далее, из матрицы X вычитаем полученную матрицу P_1, и для полученной матрицы уклонений $X_1=X-P_1$ вновь ищем наилучшее приближение P_2этого же вида и т. д., пока, например, норма X_k не станет достаточно малой. В результате получили итерационную процедуру разложения матрицы X в виде суммы матриц ранга 1, то есть $X=P_1+P_2+\ldots+P_q$ $(P_l=b^l\otimes a^l)$. Полагаем $\sigma_l=\|a^l\|\|b^l\|$ и нормируем векторы a^l , b^l : $a^l:=a^l/\|a^l\|;\ b^l:=b^l/\|b^l\|$. В результате получена аппроксимация сингулярных чисел σ_l и сингулярных векторов (правых — a^l и левых — b^l).

К достоинствам этого алгоритма относится его исключительная простота и возможность почти без изменений перенести его на данные с пробелами, а также взвешенные данные.

Back to top

Существуют различные модификации базового алгоритма, улучшающие точность и устойчивость. Например, векторы главных компонент a^l при разных l должны быть

ортогональны «по построению», однако при большом числе итерации (большая размерность, много компонент) малые отклонения от ортогональности накапливаются и может потребоваться специальная коррекция a^l на каждом шаге, обеспечивающая его ортогональность ранее найденным главным компонентам.

2. Метод Якоби.

Метод Якоби (вращений)

На каждом шаге вычисляется вращение Якоби J, с помощью которого матрица $G^\top G$ неявно пересчитывается в $J^\top G^\top G J$; вращение выбрано так, чтобы пара внедиагональных элементов из $G^\top G$ обратилась в нули в матрице $J^\top G^\top G J$. При этом ни $G^\top G$, ни $J^\top G^\top G J$ не вычисляются в явном виде; вместо них вычисляется матрица GJ. Поэтому алгоритм называется методом односторонних вращений.

Пусть a_{ij} - элемент, расположенный в i-ой строке и j-ом столбце матрицы $G^\top G$.

$\tau = (a_{jj} - a_{kk})/(2 \cdot a_{jk})$

$t = \frac{sign(\tau)}{|\tau| + \sqrt{1+\tau^2}}$. При $\tau = 0$ считаем $sign(\tau) = 1$, то есть $\theta = \frac{\pi}{4}$.

$c = \frac{1}{\sqrt{1+t^2}}$, где $c = cos\theta$

$s = c \cdot t$, где $s = sin\theta$

$G = G \cdot R(j, k, \theta)$

$J = J \cdot R(j, k, \theta)$

Здесь $R(j, k, \theta)$ — матрица вращений Якоби, которая имеет следующий вид:

Одностороннее вращение Якоби в координатной плоскости i, j вычисляется в том случае, если элемент a_{ij} удовлетворяет условию: $|a_{ij}| \geq \varepsilon \sqrt{a_{ii} a_{jj}}$.

Назад
Сравнение итерационного метода Якоби и прямого метода Гаусса

Вперёд
3.2 Проблема собственных значений

Back to top